Egg segmentation and tracking

Take-home assignment for Nash Robotics. Author: Kunal.

Code (GitHub)	https://github.com/kunalsharma0709/egg-seg
Tracked video (Google Drive)	https://drive.google.com/file/d/1Le9tQuM2JbNpxv25Q5JKKlrfJSdOELYF/view?usp=sharing
Dataset	Roboflow Universe, fire-bf7qe/egg-vzhtr, version 3: https://universe.roboflow.com/fire-bf7qe/egg-vzhtr/dataset/3
Setup and run commands	README.md in the repository (every task has one command)
Experiment log	experiments/experiment_log.md (also summarised in Section 3)

1. Summary

Task	What was done	Headline result
1	Grounding DINO (tiny) finds eggs from the text prompt 'egg.', SAM ViT-B turns each box into a mask. One instance id per egg.	160 images segmented: 137 with 1 egg, 16 with 2, 7 with 3. 2 images judged unusable and excluded from the split.
2	Group-wise 4:1 dev/eval split (dHash near-duplicate groups). Classical algorithm: GrabCut with a 2% margin, morphology, blob filters. Nine logged trials.	Dev (126 images): IoU 0.600, Dice 0.697, count accuracy 0.841.
3	Task 1 masks as reference, Task 2 masks as prediction. Pixel, instance (Hungarian matching) and boundary metrics, plots, failure analysis.	Eval (32 images, run once): IoU 0.560 (95% CI 0.471-0.649), Dice 0.677, instance F1 0.522, count accuracy 0.875.
4	The Task 2 algorithm fails on the video, so the detector was replaced by a white-egg colour gate plus watershed. Tracker: Hungarian matching on mask IoU and centroid distance.	2 eggs tracked with 3 ids over 60 s (9 ids with a short memory). One re-identification failure.

The scores are measured against SAM-based masks, not human labels. Task 1 errors therefore count as errors of the classical method in Task 3. Where something was a guess and not checked, the text says so.

2. Task 1: segmentation with a pretrained model

2.1 Data

The 160 images (640 x 640) come from the Roboflow download in 'Folder Structure' format (train, valid and test folders). They were merged into one folder; the Roboflow split is not used, because Task 2 needs its own leakage-aware split. Filenames carry a Roboflow hash, and the group id (filename before '.rf.') was unique for all 160 images, so there are no exact duplicates by name.

2.2 Method

First attempt: SAM automatic mode. The Hugging Face mask-generation pipeline (point grid) plus filters on size, solidity, aspect ratio and mask NMS. On 5 test images it returned 0, 2, 2, 1, 1 masks although most images contain one egg. Debug output showed that the egg mask is large (14 to 35% of the image) and most other masks are fragments (shadows, tray, texture). Tightening the filters gave one mask per image, but several masks were not accurate. SAM in automatic mode does not know what an egg is, so I changed the method.

Final: Grounded-SAM. Grounding DINO (IDEA-Research/grounding-dino-tiny) is given the text prompt `egg.` and returns boxes with scores. Boxes that are too large are removed, box NMS is applied, and at most 3 boxes are kept (the dataset has at most 3 eggs). Each box is a prompt for SAM ViT-B (facebook/sam-vit-base, `multimask_output=False`), which returns one mask per egg. Tiny masks and duplicate or contained masks are dropped. The result is a uint16 label image per photo: 0 is background, 1 to N are the eggs (so Task 3 and Task 4 can work per egg).

Parameter	Default	Meaning
box_thr / text_thr	0.30 / 0.25	minimum DINO box score / text match score
fallback_thr	0.15	if no box passes box_thr, detect again at this threshold
max_box_frac	0.80	boxes larger than this fraction of the image are dropped (stops tray/background boxes)
box_nms_iou	0.5	duplicate boxes on the same egg
max_eggs	3	most eggs in any image of this dataset
dedupe_iou / contain_thr	0.7 / 0.8	drop a mask that overlaps or sits inside a kept mask
min_mask_frac	0.005	drop masks smaller than 0.5% of the image

2.3 Results and the problems found by looking at the overlays

First full run: 12 images without a mask, 124 with one egg, 17 with two, 7 with three. After adding the fallback threshold, box NMS and the containment test: 9, 126, 18, 7. The 9 images without a mask were all extreme close-ups in which one egg fills much of the frame; `max_box_frac=0.8` removed its box. Re-running these with `--max_box_frac 1.0 --box_thr 0.2` gave one mask each. Counted from the final mask files: **137 images with 1 egg, 16 with 2 eggs, 7 with 3 eggs** (160 in total).

I checked overlays on contact sheets: random images, the images that had no mask in the first run, and the multi-egg images (in that check about 21 to 22 of 25 looked clean). Problems found:

- **train_not_damaged_139**: the second mask sat on a red container. Raising box_thr to 0.4 removed it.
- **train_not_damaged_42**: two egg-like masks, one with a jagged lower-right boundary. Kept as it is.
- **train_not_damaged_157** (large egg in a hand) and **train_not_damaged_39** (fruit-print plate): wrong masks (Figure 1; the mask in 157 lies on the background around the egg).
- **Excluded:** two images whose masks were judged unusable are listed in outputs/exclude.txt. They stay in outputs/task1/masks but are left out of the dev/eval split.
- **train_not_damaged_116** (egg not visible, mask on a tray) was noticed later and was not excluded, because that would change the split after experiments had started. It is counted as an error in Task 3.

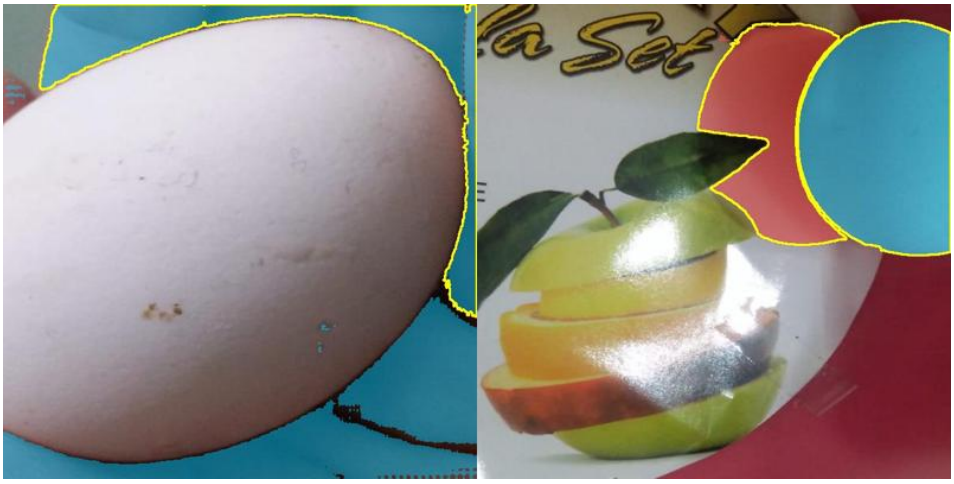


Figure 1. Task 1 failure cases found in the visual check, before any fix or exclusion (left: train_not_damaged_157, right: train_not_damaged_39).

Limitation. The masks were not verified one by one, and they are not human labels. Because Task 3 treats them as ground truth, their errors lower the scores of the classical method.

3. Task 2: split and classical segmentation

3.1 Dev/eval split (4:1, with reduced leakage)

Near-duplicate or same-scene photos must stay on one side of the split, otherwise tuning on dev leaks into eval. Steps:

- Remove the 2 excluded images (158 remain). The Roboflow train/valid/test split is ignored.
- Compute a 64-bit dHash per image (grayscale 9 x 8, compare neighbouring pixels). Join images with Hamming distance up to 6 into one group (union-find).
- Hamming distance 10 was tried first: white eggs have similar hashes, so one group chained 6 images from 4 different scenes. I changed the threshold to 6.
- At 6, three images of one leaves scene (different crop and zoom) were split across dev and eval. They were merged by hand into one group (train_94, valid_75, valid_9).
- Groups are stratified by their largest egg count (1, 2 or 3) and about 20% of the groups per stratum go to eval. Seed 42. The split file is data/split.csv.

Result: 151 groups (6 with more than one image). **Dev 126 images** (108 with 1 egg, 12 with 2, 6 with 3), **eval 32 images** (28 / 3 / 1). All parameters were tuned on dev only. Eval was run once, after the algorithm was frozen. *Limitation:* dHash is not robust to crop and zoom, so some near-duplicates may still be on both sides; the eval split has only one 3-egg image.

3.2 Final algorithm

- **GrabCut** (OpenCV, 5 iterations) initialised with a rectangle that leaves a 2% margin at every image border. Pixels outside the rectangle are certain background, inside are probable foreground. GrabCut fits a colour mixture model (5 Gaussians) to foreground and background, then solves a graph min-cut that also prefers smooth labels; this is repeated.
- **Morphology:** open then close with a 7 x 7 ellipse (removes specks, fills small holes).
- **Blob filters** on connected components: area at least 0.5% of the image; keep the 3 largest; drop blobs smaller than 30% of the largest; drop blobs with solidity (area divided by convex hull area) below 0.8, except that if this removes every blob, the blobs from before the solidity filter are kept.
- **Output:** filled contours, one label per blob (same format as Task 1).

3.3 Experiment log

Every trial was measured on the dev split against the Task 1 masks: mean per-image pixel IoU, Dice, and count accuracy (predicted number of eggs equals the reference number). To see why an image scored badly I used a diagnostic: the ratio of predicted to reference area for images with IoU between 0.1 and 0.8 (over-segmented if the ratio is above 1.3, under-segmented below 0.77, otherwise close). It compares total area, not location, so a shifted mask can still look 'close'.

Trial	What changed (parameters)	Why	Dev IoU / Dice / count acc	Decision
T1	Gray, Gaussian blur 5, Otsu threshold, flip the mask if over 50% of the border is foreground, open + close (7 x 7), 3 largest blobs	Simplest baseline	0.510 / 0.600 / 0.548	Replaced. Worst cases: white egg on white background, flip on close-ups, textured background chosen instead of the egg (Figure 2, left)
T2	Otsu replaced by GrabCut with rectangle margin 0.10, 5 iterations; same post-processing	Remove the polarity heuristic, use a colour model	0.536 / 0.649 / 0.802	Kept. 13 images IoU below 0.1, 24 at 0.8 or above, 89 in between (38 under-, 18 over-segmented, 33 close)
T3b	Margin 0.10 to 0.05	Eggs touching the frame edge were cut: everything outside the rectangle is certain background	0.572 / 0.676 / 0.778	Discarded, dominated by T3a (under 15, over 27, close 35)
T3a	Margin 0.10 to 0.02	Same reason, stronger	0.605 / 0.706 / 0.770	Kept. Under-segmented 38 to 8, over-segmented 18 to 32: a larger rectangle lets more background into the colour model
T4	T3a plus: drop blobs smaller than 30% of the largest (rel_area 0.3)	Count accuracy low; extra small blobs suspected	0.605 / 0.706 / 0.817	Kept. Count accuracy +0.047, IoU unchanged (removed blobs were small)
T5	T4 plus: drop blobs with solidity below 0.9	Failure blobs looked irregular, eggs are convex	0.410 / 0.459 / 0.540	Discarded. 49 of 126 predictions became empty (every blob removed); checked by counting images with no predicted egg
T6	T5 plus: if nothing remains, look for convex blobs in the inverted GrabCut mask	Hypothesis: GrabCut swapped foreground and background in some images	0.428 / 0.487 / 0.651	Discarded. 30 predictions still empty and IoU low: hypothesis not supported
T7a	T5 plus fallback: if the solidity filter removes every blob, keep the blobs from before it	Fix the empty predictions of T5	0.594 / 0.689 / 0.841	Discarded, IoU lower than T7b
T7b	T7a with solidity threshold 0.8	Less strict filter	0.600 / 0.697 / 0.841	**Final.** Versus T4: count accuracy +0.024, IoU -0.005 (within noise on 126 images). Chosen because Task 4 needs the right number of eggs; T4 would also be defensible

Notes. (1) After refactoring the code to cache raw GrabCut masks, T3a and T4 were re-run and gave the same numbers. (2) A GrabCut variant with a certain-foreground seed in the image centre was considered and dropped before running: in the worst images the egg is at the frame edge, not in the centre. (3) Final configuration: `--method grabcut --margin 0.02 --rel_area 0.3 --min_sol 0.8 --fallback`.

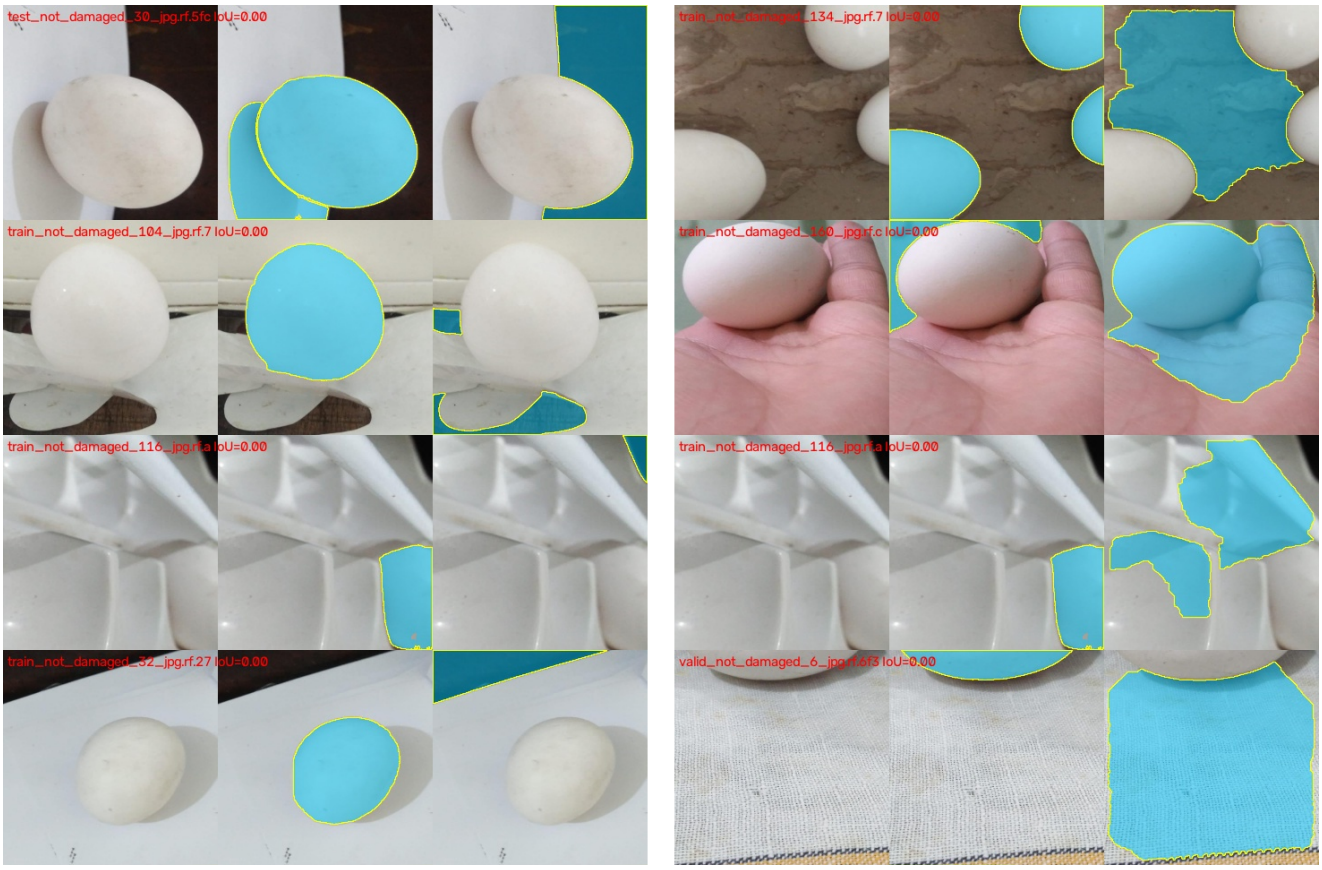


Figure 2. Dev split, four of the worst images for T1 (left) and for T4 (right). Columns in each block: image, Task 1 reference mask, prediction. T1 (Otsu) lands on shadows, a dark corner, text and cloth. T4 (GrabCut) misses eggs at the frame edge and segments a large background region instead; in train_160 the hand is added to the egg.

4. Task 3: evaluation and analysis

Reference: Task 1 masks. Prediction: final Task 2 masks. Pixel metrics use the union of all eggs per image and are averaged over images. Instance metrics match predicted and reference eggs one-to-one with the Hungarian algorithm on mask IoU and count a match when IoU is above 0.5. Boundary F1 counts a boundary pixel as correct when the other boundary is within 3 pixels. The confidence interval is a bootstrap over images (1000 resamples).

Metric	Eval (32 images, run once)	Dev (126 images)
Pixel IoU (95% CI)	0.560 (0.471 to 0.649)	0.600 (0.549 to 0.649)
Dice	0.677	0.697
Pixel precision / recall	0.642 / 0.786	0.681 / 0.758
Boundary F1 (3 px)	0.308	0.341
Instance precision / recall / F1	0.562 / 0.486 / 0.522	0.667 / 0.573 / 0.616
Count accuracy	0.875	0.841
Images with IoU below 0.1 / at least 0.8	1 / 8	12 / 43
IoU, images with 1 egg	0.594 (28 images)	0.631 (108 images)
IoU, images with 2 or 3 eggs	0.326 (4 images)	0.414 (18 images)

4.1 Quantitative findings

- **Over-segmentation dominates.** Pixel precision is lower than recall on both splits: the mask usually contains the egg plus extra background, and eggs are rarely missed completely.
- **Eval is slightly below dev, but the confidence intervals overlap** (0.471 to 0.649 and 0.549 to 0.649), so with 32 images I do not claim a real difference.
- **Count accuracy hides location errors.** It is 0.875 on eval, but instance F1 is only 0.522, because a mask in the wrong place still gives the right count.
- **Boundary F1 is low** (0.31 on eval). Likely causes (not tested separately): GrabCut boundaries are jagged while the SAM reference is smooth, and masks that include extra background have wrong boundaries.

- **The IoU distribution is wide** (Figure 4): 8 of 32 eval images reach IoU 0.8 or more, while 9 are below 0.3. The algorithm either finds the egg cleanly or attaches a large region.
- **Multi-egg images are worse** (0.326 on 4 eval images, 0.414 on 18 dev images). The eval sample is too small for firm conclusions.

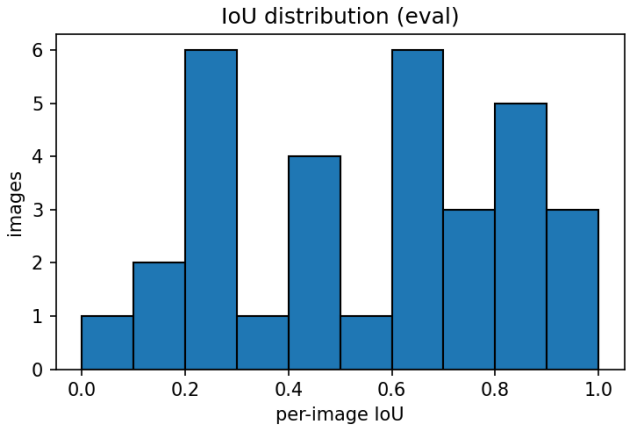


Figure 4. Per-image IoU on the eval split.

4.2 Qualitative analysis

Figure 5 shows the eight eval images with the lowest IoU as error maps (green: correct, red: false positive, blue: false negative). My reading of the failures:

- **Egg found, but a colour-similar background is attached** (high recall, precision 0.16 to 0.33): pink plate (train_22), steel pan (train_145), cloth (train_95), tray (train_121). GrabCut's colour model cannot separate a white egg from a pale surface.
- **Eggs at the frame edge are missed** while background is segmented (train_105, train_121). A likely cause is that the border strip is certain background, so the colour of the cut egg is learned as background. This was suggested by the margin experiment (T3) but not tested directly.
- **Shadow merged with the egg** (valid_45).
- **White egg on white tray, or a close-up** (train_42, train_44): the boundary is ambiguous and the reference mask itself is uncertain.

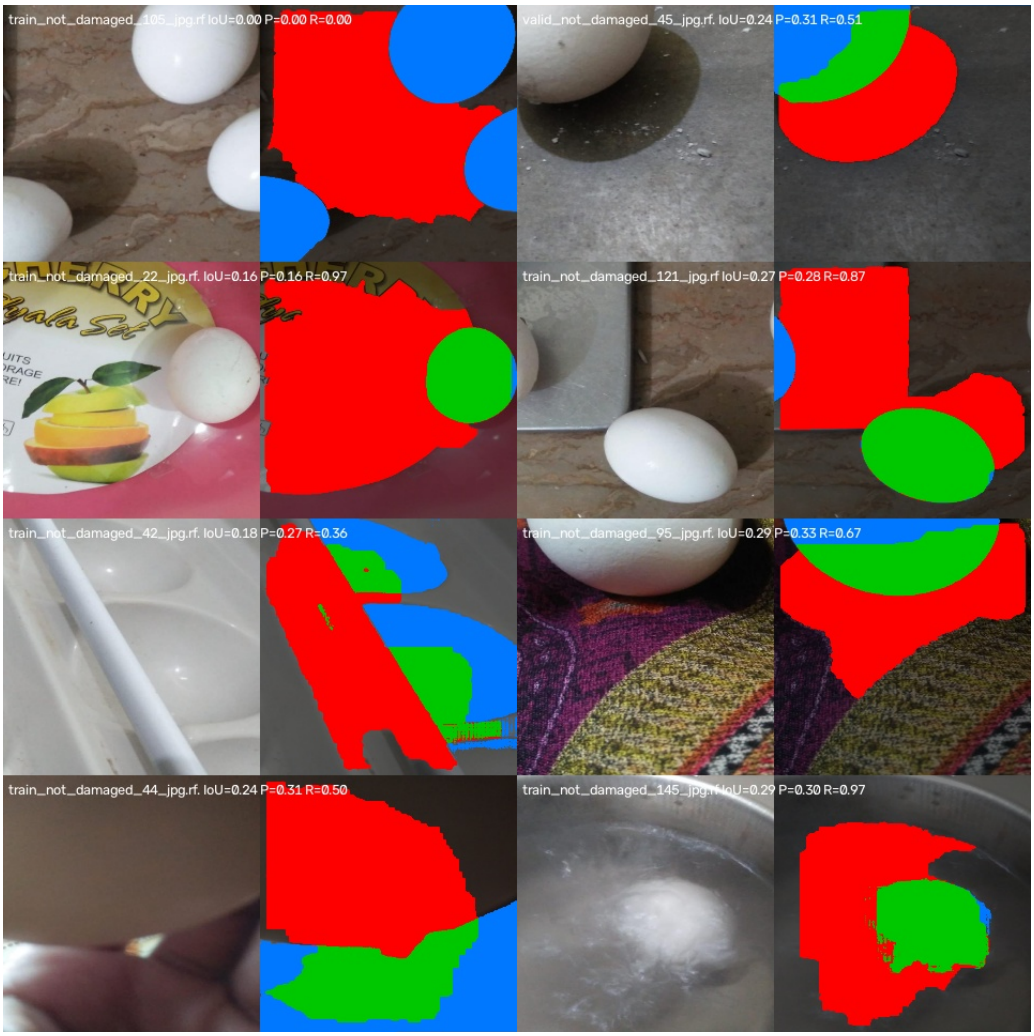


Figure 5. Eval split, eight lowest-IoU images. Left column, top to bottom: train_105, 22, 42, 44; right column: valid_45, train_121, 95, 145. Each pair: image and error map.

Multi-egg images. All four multi-egg eval images were under-counted (one predicted egg each), for different reasons: train_105 and train_121 are the frame-edge case above, train_42 is the white-on-white scene, and valid_115 (IoU 0.85) shows two touching eggs merged into one blob. Pixel IoU does not penalise a merge, but instance recall does, which is why instance recall (0.486) is much lower than pixel recall (0.786).

4.3 Limitations and improvements

- The reference masks are model output. They contain errors (for example train_116), and the dev/eval samples are small, especially for multi-egg images.
- **Improve border handling:** treat the border strip as probable, not certain, background, or pad the image before GrabCut.
- **Reject over-segmented masks with a shape prior:** fit an ellipse to each blob and use the fit error; or run GrabCut with several initialisations and pick the most egg-like result.
- **Split touching eggs** with a distance-transform watershed (used in the video detector, Section 5).
- **Use Lab colour or texture features** to separate white eggs from pale surfaces.
- **Smooth boundaries** with a convex hull or ellipse fit, since eggs are convex.
- These were not run on the eval split, to keep eval untouched after the algorithm was frozen.

5. Task 4: video tracking

The video (Mocomi, 60 s, 24 fps, 640 x 360) has a title card, then a table with two eggs (raw and hard-boiled) and a hand that lifts, moves and spins them, then text cards and an end card (Figure 6). The task is to track the two eggs separately. Output video: see the Drive link on page 1.



Figure 6. One frame every 3 s. The eggs are on screen from about 11 s to 40 s; the Mocomi logo is in the bottom-right corner throughout.

5.1 The Task 2 algorithm does not work on this video

Running the Task 2 algorithm per frame gave foreground regions on the green title card (it always returns some foreground) and one id on the logo. In a debug run on one frame per second, every frame of the egg scene (10 to 39 s) produced a single blob covering 38 to 63% of the frame, which an area limit rejected, so no egg was output. This debug run used an earlier version of the tracking script. My explanation, not tested further: the near full-frame rectangle and the dark table make GrabCut return the large dark region.

5.2 Modification: white-egg detector

Sampled pixel values in one frame (HSV, 0 to 255): egg saturation about 0 and value about 245, table about 95 and 94, shirt about 34 and 149. Eggs are therefore bright and unsaturated. Per frame (resized to width 320, every second frame, 720 frames processed):

- Threshold value above 190 and saturation below 45; set the logo corner to zero; open and close (5 x 5).
- **Distance-transform watershed** to split touching eggs: markers are the distance-map regions above 55% of the blob's maximum; a blob with a single marker is not split. (The two eggs touch from 12 to 18 s.)
- Blob filter: area between 0.4% and 35% of the frame, centroid below 38% of the frame height (removes the wall and shirt), solidity at least 0.75.

5.3 Tracker

Each frame gives a list of masks. A cost matrix between the tracks of the previous frame and the new masks is built as (1 - mask IoU) + centroid distance divided by the image diagonal, and solved with the Hungarian algorithm. A pair is accepted if IoU is at least 0.2 or the centroid distance is at most 15% of the diagonal. A mask without a match starts a new id. A track that is not matched is kept for `max_age` processed frames (so it can be matched again after an occlusion), then deleted. The output video shows the mask and id of each egg; a CSV logs frame, time, id and centroid.

5.4 Results

Version	max_age	Unique ids (2 eggs)	Remarks
v1	5	9	Every hand interaction broke the track (egg hidden for several seconds)
v2 (final)	100	3	Tracks survive occlusions; one extra id remains

v2 id	Active (s)	Frames	Mean x (px of 320)	Reading
1	9.25 to 31.25	245	77	right egg of the first scene (hard-boiled)
2	9.83 to 41.75	253	36	left egg of the first scene (raw)
3	22.5 to 41.58	146	205	extra id: an egg put down on the right

Failure. Around 21 to 24 s an egg is lifted by the hand and put down about 170 px to the right. This is outside the matching gate, so it gets a new id (id 3). The mapping of ids to eggs in the table is my reading of the positions and the contact sheet. I did not check frame by frame whether the ids of the two eggs were swapped after the hand interactions. The hand also hides an egg for several seconds around 27 to 33 s; the detector then finds only part of it or nothing.

Limitations. There is no ground truth for the video, so the result is qualitative. The detector thresholds were chosen by looking at about ten frames of this same video and rely on white eggs on a dark table; this is a video-specific modification and was not tested on the dataset. The tracker has no motion model and no appearance feature (the raw and the boiled egg look the same). Output is 12 fps because every second frame is processed. Fast spinning with motion blur was not analysed separately.

6. Use of AI tools

I used Claude (Anthropic) for drafting code, debugging, and discussing methods and metrics, and for help with the layout of this report. I ran every experiment, looked at the overlays and failure sheets myself, and made the choices recorded above (method change in Task 1, split threshold, the margin and filter experiments, which trials to keep, the video modification). I can explain each step of the algorithms.

7. Reproduction

Full instructions are in README.md. Condensed (Windows PowerShell; Python 3.10, torch 2.14.1 with CUDA 12.6, transformers 5.20.2):

```
python -m venv .venv ; .venv\Scripts\Activate.ps1
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt ; pip install -e .
# put the Roboflow export in data/raw (train, valid, test), then:
python -m eggseg.data                                  # merge images, manifest
python -m eggseg.gsam_seg                              # Task 1 masks
python -m eggseg.split                                 # Task 2 dev/eval split
python -m eggseg.classical_seg --method grabcut --margin 0.02 --rel_area 0.3 --min_sol 0.8 --fallback `
    --split dev --out outputs/task2/final_dev
python -m eggseg.classical_seg --method grabcut --margin 0.02 --rel_area 0.3 --min_sol 0.8 --fallback `
    --split eval --out outputs/task2/final_eval
python -m eggseg.evaluate --split eval --pred outputs/task2/final_eval     # Task 3
python -m eggseg.track --out outputs/task4/tracked_final.mp4               # Task 4 (video in data/video/egg.mp4)
```

All trials of Task 2 can be reproduced with the commands in experiments/experiment_log.md. The split file (data/split.csv) is committed; raw images and the source video are not.